

PROJETO INCREMENTAL DA DISCIPLINA

Bancos de Dados NoSQL: Persistência e Distribuição de Big Data

Do problema real ao sistema: introdução, contexto e regras do projeto

Introdução

Vivemos em uma era de Big Data: aplicações modernas — redes sociais, e-commerce, streaming, aplicativos de transporte, sensores de IoT — geram e consultam volumes de dados que crescem em volume, velocidade e variedade muito além do que os bancos de dados relacionais foram projetados para suportar quando surgiram, nos anos 1970. Ao longo desta primeira metade da disciplina, vocês estudaram por que esse descompasso acontece e conheceram duas famílias de tecnologias criadas especificamente para resolvê-lo: os bancos Chave-Valor e os bancos Wide-Column.

O projeto incremental é o espaço da disciplina reservado para transformar essa teoria em prática. Ele reproduz, em escala reduzida, o mesmo exercício que um engenheiro ou arquiteto de dados enfrenta no mundo real: diante de um problema concreto, decidir — com argumentos técnicos, não por preferência — qual tecnologia de persistência é a mais adequada, e então implementá-la. Este documento situa o projeto dentro do conteúdo já estudado e detalha o que se espera da primeira entrega, o Marco 1.

O que é NoSQL?

O termo NoSQL foi cunhado em 1998 e popularizado a partir de 2009 para descrever uma família de bancos de dados que abrem mão de parte das garantias do modelo relacional clássico em troca de escalabilidade horizontal, alta disponibilidade e esquemas flexíveis. Não é uma tecnologia única, mas um conjunto de abordagens que compartilham a mesma motivação: sistemas relacionais têm dificuldade de escalar horizontalmente porque operações como joins e transações distribuídas ficam cada vez mais custosas conforme os dados são espalhados por múltiplos servidores.

Dois conceitos estruturam praticamente toda decisão de arquitetura nesse universo — e vocês já dominam os dois:

- **Teorema CAP** — em um sistema distribuído não é possível garantir simultaneamente Consistência, Disponibilidade e Tolerância a Partição; como a tolerância a partição é praticamente obrigatória em qualquer sistema real, a escolha de fato se dá entre priorizar Consistência (sistemas CP) ou Disponibilidade (sistemas AP).
- **ACID vs. BASE** — enquanto bancos relacionais buscam garantias fortes (Atomicidade, Consistência, Isolamento, Durabilidade), muitos bancos NoSQL adotam o modelo BASE — Basically Available, Soft state, Eventually consistent —, aceitando inconsistências temporárias em troca de disponibilidade e desempenho.

As quatro famílias de bancos NoSQL

- **Chave-Valor** — dados acessados por uma chave única (Redis, Dynamo, Riak, Voldemort). Já estudada — Unidade 2.
- **Wide-Column (Colunar)** — dados organizados em famílias de colunas, otimizados para grandes volumes analíticos (BigTable, Cassandra, HBase). Já estudada — Unidade 3.
- **Documentos** — dados semiestruturados em JSON/BSON (MongoDB, CouchDB). Vocês vão estudar a partir da próxima unidade.
- **Grafos** — dados centrados em relacionamentos (Neo4j). Também vem nas próximas unidades.

O Marco 1 do projeto pede que vocês apliquem as duas primeiras famílias — Chave-Valor e Wide-Column — e os fundamentos de sistemas distribuídos (CAP, ACID/BASE, sharding e replicação) a um problema real escolhido pelo grupo.

NoSQL no mundo real

As tecnologias que vocês estudaram não nasceram em ambiente acadêmico — surgiram porque empresas de internet em escala global esbarraram, na prática, no limite dos bancos relacionais. Conhecer o problema original por trás de cada tecnologia ajuda a entender por que ela foi desenhada da forma como foi:

Empresa / Tecnologia	Família	Problema real que motivou a criação
Amazon — Dynamo	Chave-Valor	Garantir que "adicionar ao carrinho" nunca falhe, mesmo sob pico de tráfego ou falha de rede.
Google — BigTable	Wide-Column	Indexar e servir bilhões de páginas e sinais de uso (Search, Analytics) em escala massiva.
Facebook — Cassandra	Wide-Column	Eliminar o ponto único de falha na busca de mensagens (Inbox Search), com escrita distribuída.
Ecossistema Hadoop — HBase	Wide-Column	Processar em lote grandes volumes de logs e cliques, usando MapReduce sobre o HDFS.

O projeto pede que o grupo faça, em pequena escala, o mesmo raciocínio: identificar um problema real, seus requisitos de consistência e disponibilidade, e só então escolher a tecnologia — não o contrário.

O pedido do trabalho

O projeto é a espinha dorsal prática da disciplina: em grupo, vocês vão desenvolver um sistema real usando tecnologias NoSQL, aplicando ao longo do semestre os conceitos vistos em cada unidade. Ele vale 40% da nota final e é entregue em duas etapas — o Marco 1 e o Marco 2 — descritas a seguir.

Visão geral

- Trabalho em grupo (até 4 alunos).
- Duas entregas ao longo do semestre: Marco 1 (modelagem inicial + apresentação) e Marco 2 (sistema completo + apresentação).
- O projeto vale 40% da nota final, dividido entre as duas entregas e a apresentação.
- O mesmo grupo e o mesmo problema atravessam as duas etapas — o Marco 2 evolui a partir do Marco 1, não é um projeto novo.

Marco 1 — Modelagem inicial

Primeira entrega do projeto. Cada grupo escolhe um problema real — pode se inspirar em uma aplicação do dia a dia, uma startup fictícia, um problema do próprio trabalho, etc. — e propõe uma primeira modelagem de dados, escolhendo e justificando a família (ou famílias) de banco NoSQL mais adequada para essa etapa inicial do sistema. A entrega precisa mobilizar tudo o que foi estudado até aqui — Unidades 1 a 3 —, não apenas o conteúdo da última aula.

O que entregar

1. Descrição do problema (2–3 páginas): o contexto da aplicação, os requisitos principais e uma estimativa do volume de dados envolvido.
2. Modelagem inicial dos dados: diagramas ou exemplos concretos de como os dados serão organizados.
3. Justificativa tecnológica: por que a família NoSQL escolhida é adequada para essa primeira parte do sistema — com argumentos técnicos, conforme o checklist abaixo.

Checklist técnico — o que a justificativa precisa mostrar

A nota do Marco 1 avalia, sobretudo, a qualidade da justificativa. Ela precisa demonstrar domínio do conteúdo de cada unidade já vista, aplicado ao problema escolhido pelo grupo:

Unidade 1 — Fundamentos de sistemas distribuídos

- **Limites do relacional** — por que o problema escolhido seria mal atendido (ou ficaria caro/lento) em um banco relacional tradicional — custo de joins e de transações distribuídas em escala.
- **Teorema CAP** — o sistema prioriza Consistência (CP) ou Disponibilidade (AP)? A escolha precisa ser justificada com base no cenário do problema, não apenas declarada.
- **ACID vs. BASE** — o sistema seguirá garantias ACID ou o modelo BASE? Por quê, dado o tipo de dado e de operação envolvidos?
- **Sharding e replicação** — proposta de estratégia de particionamento (hash, range ou directory-based) e de replicação (Master-Slave ou Master-Master) compatível com o volume estimado.

Unidade 2 — Chave-Valor

- **Uso do modelo** — se o grupo optar por (ou incluir) um banco Chave-Valor, quais dados do problema se encaixam nesse modelo (cache, sessão, contadores, carrinho, ranking)?
- **Técnica citada** — referência a pelo menos um mecanismo estudado — estruturas do Redis (strings, listas, sets, hashes, sorted sets), Consistent Hashing ou Vector Clocks do Dynamo — aplicado ao caso do grupo.

Unidade 3 — Wide-Column

- **Uso do modelo** — se o grupo optar por (ou incluir) um banco Wide-Column, apresentar uma modelagem orientada a consultas (query-first), pensando primeiro nas consultas da aplicação.
- **Técnica citada** — definição de Column Families e, se usarem Cassandra, de partition key e clustering columns coerentes com os acessos previstos.

Grupos podem propor uma solução que combine mais de uma família (ex.: Chave-Valor para cache + Wide-Column para o volume principal de dados) — isso é incentivado e deve aparecer explicitamente na justificativa.

Como funciona em aula

Cada grupo entrega a documentação e faz uma apresentação breve em sala, apenas para o professor verificar o andamento — não é uma banca formal. É o momento de alinhar expectativas antes de avançar para o sistema completo.

Como entregar

- **Formato** — um único PDF, nomeado no padrão Marco1_NomeDoGrupo_Campus.pdf (ex.: Marco1_DataFlow_Taguatinga.pdf).
- **Canal** — upload no AVA da disciplina, na atividade “Marco 1 — Projeto Incremental”.
- **Prazo** — até 23h59 do dia de entrega do respectivo campus — ver Calendário das entregas, ao final deste documento.
- **Apresentação** — fala breve de 3 a 5 minutos em sala, no mesmo dia da entrega, com verificação do professor.

Exemplo resolvido — Marco 1: “CEUB Delivery”

O exemplo abaixo é fictício e serve apenas como modelo de formato e de profundidade esperada — não deve ser copiado como problema do grupo. Ele percorre os três entregáveis do Marco 1 e mostra, na prática, como o checklist técnico da seção anterior se traduz em texto.

1. Descrição do problema

O “CEUB Delivery” é um aplicativo fictício de delivery de comida que conecta alunos aos restaurantes do entorno dos campi. O sistema precisa suportar três operações centrais: (i) um carrinho de compras com atualização em tempo real durante a montagem do pedido; (ii) o status do pedido evoluindo em etapas (recebido → em preparo → a caminho → entregue); e (iii) um histórico de pedidos por restaurante, usado por um painel de analytics que os donos dos restaurantes parceiros acessam para acompanhar vendas.

O tráfego é fortemente concentrado: nos horários de almoço e jantar, o sistema estima um pico de aproximadamente 2.000 pedidos por dia, concentrados em cerca de 90 minutos por refeição — o que equivale a

picos de dezenas de escritas por segundo em carrinho e status, muito acima do tráfego médio do restante do dia. A base projetada é de 5.000 usuários ativos e 120 restaurantes parceiros no primeiro semestre de operação, com expectativa de crescimento trimestral.

2. Por que não um banco relacional aqui

- **Custo de joins sob pico** — montar a tela do carrinho normalmente exigiria juntar usuário, itens, preços e restaurante em uma única consulta; sob pico de almoço, repetir esse join a cada atualização de carrinho, para milhares de usuários simultâneos, tende a saturar um único servidor relacional.
- **Escrita muito concentrada no tempo** — 90 minutos concentram grande parte do volume diário — o sistema precisa escalar horizontalmente durante a janela de pico e não apenas verticalmente, o que é mais natural em bancos distribuídos desde a concepção.
- **Padrão de acesso previsível** — as consultas do painel de analytics (“pedidos de um restaurante em um intervalo de datas”) são conhecidas de antemão, o que favorece uma modelagem orientada a consultas em vez do modelo genérico de tabelas e joins do relacional.

3. Modelagem inicial dos dados

O grupo optou por combinar duas famílias — Chave-Valor para o carrinho (dado volátil, de acesso único por usuário) e Wide-Column para o histórico de pedidos (dado durável, consultado por restaurante e período):

```
// Carrinho – Redis (Hash com TTL de 30 min)
carrinho:usuario:8842
  item:301 -> { qtd: 2, preco: 18.90, nome: "Marmita fit" }
  item:455 -> { qtd: 1, preco: 12.50, nome: "Suco natural" }
TTL: 1800s (expira se o pedido não for finalizado)

// Ranking de itens mais pedidos por restaurante – Redis (Sorted Set)
ranking:restaurante:57 -> ZADD item:301 count

-- Histórico de pedidos – Wide-Column (estilo Cassandra/CQL)
CREATE TABLE pedidos_por_restaurante (
  restaurante_id int,
  data_pedido timestamp,
  pedido_id uuid,
  usuario_id int,
  status text,
  valor_total decimal,
  PRIMARY KEY (restaurante_id, data_pedido)
) WITH CLUSTERING ORDER BY (data_pedido DESC);
```

A partition key (restaurante_id) determina o nó responsável pelos dados de cada restaurante, e a clustering column (data_pedido, em ordem decrescente) já entrega os pedidos mais recentes primeiro — exatamente a consulta que o painel de analytics faz com mais frequência.

4. Justificativa tecnológica

- **Limites do relacional** — detalhado no item 2 acima: custo de joins repetidos sob pico e necessidade de escalabilidade horizontal na janela de almoço/jantar.
- **Teorema CAP** — o carrinho prioriza Disponibilidade (AP) — perder um “adicionar ao carrinho” por indisponibilidade é pior do que servir um estado momentaneamente desatualizado. O histórico de pedidos também tolera consistência eventual entre réplicas, já que o painel de analytics não precisa refletir o último milissegundo.
- **ACID vs. BASE** — o sistema segue o modelo BASE: soft state no carrinho (expira automaticamente em 30 min) e consistência eventual no histórico. Os dados de pagamento, que exigiriam garantias ACID mais fortes, ficam fora do escopo deste Marco 1 e seriam tratados por um serviço à parte.

- **Sharding e replicação** — particionamento directory-based por restaurante_id no histórico (cada restaurante concentra seus próprios dados em partições previsíveis) e replicação Master-Slave para suportar a leitura intensa do painel de analytics sem sobrecarregar o nó de escrita.
- **Chave-Valor (Unidade 2)** — Redis Hash para o carrinho com TTL, e Sorted Set para o ranking de itens mais pedidos — ambos casos de uso clássicos do modelo chave-valor, já vistos na Aula 5.
- **Wide-Column (Unidade 3)** — modelagem query-first pensando na consulta mais frequente do negócio (“pedidos de um restaurante por período”), com partition key e clustering column definidos a partir dela — não a partir das entidades, como se faria no relacional.

5. O que este exemplo demonstra

Note o paralelo direto com o caso do Dynamo da Amazon apresentado na seção “NoSQL no mundo real”: assim como a Amazon não podia deixar a operação de carrinho falhar, o CEUB Delivery aplica o mesmo raciocínio — disponibilidade antes de consistência forte — para um problema em escala muito menor, mas com a mesma lógica de decisão. No Marco 2, esse mesmo grupo evoluiria o sistema adicionando, por exemplo, um banco de Documentos para o cardápio de cada restaurante (dado semiestruturado e mutável) — mas isso já é conteúdo das próximas unidades.

Marco 2 — Sistema completo e apresentação final

Segunda e última entrega. O grupo evolui o que foi proposto no Marco 1 até um sistema de persistência completo, agora integrando as diferentes famílias NoSQL trabalhadas ao longo do curso, com documentação e demonstração funcionando.

O que entregar

1. Documentação final completa: a modelagem de dados e as decisões de arquitetura tomadas pelo grupo, com justificativa.
2. Sistema funcional: pronto para demonstração ao vivo (ou em vídeo, se necessário).
3. Apresentação de 10 a 12 minutos: problema, arquitetura adotada, demonstração e lições aprendidas.

Como funciona em aula

A aula do Marco 2 é um seminário: cada grupo apresenta seu projeto para a turma. Os demais grupos avaliam a apresentação dos colegas usando uma rubrica (avaliação por pares) e o professor dá a devolutiva final.

Como o projeto entra na nota final

Instrumento	Peso	O que é
Provas (P1 + P2)	40%	Duas avaliações teórico-práticas individuais.
Projeto Incremental	40%	Marco 1 + Marco 2 + Apresentação Final, em grupo.
Exercícios / Listas	20%	Atividades semanais de fixação.

Média Final = $0,4 \times \text{Provas} + 0,4 \times \text{Projeto} + 0,2 \times \text{Exercícios}$

Calendário das entregas

Datas de entrega do Marco 1, por campus (aula que fecha a Unidade 3):

- **Campus Taguatinga** — 29/09.
- **Campus Asa Norte** — 30/09.

A data do Marco 2 + Apresentação Final será confirmada no cronograma da disciplina; a ordem é fixa:

-
- Marco 2 + Apresentação Final — entregue na penúltima aula do semestre, antes da Prova 2.
-

Dúvidas sobre o escopo do projeto podem ser tiradas com o professor a qualquer momento do semestre — quanto antes o grupo alinhar o problema escolhido, melhor.